TIMELINE EDGE AI 2026

Mở đơn đăng ký	Mở vòng Warm-up, kiểm thử hệ thống thi đấu	Kickoff - Training 1	Thi đấu vòng loại	Công bố đội thi vào Chung kết + Phát Kit	Chung Kết
24/10/2026 - 29/10/2026	01/11/2026 - 07/11/2026	8/11/2026	09/11/2026 - 18/11/2026	20/11/2026	29/11/2026

TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

EDGE AI CHALLENGE 2026 LẦN 2

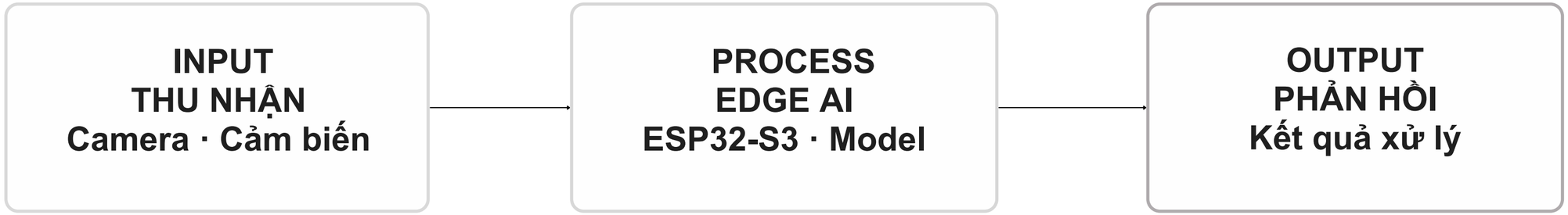
TRAINING 1

Object Detection & Intent TextCNN trên ESP32-S3

PIONEER – Tháng 11/2026

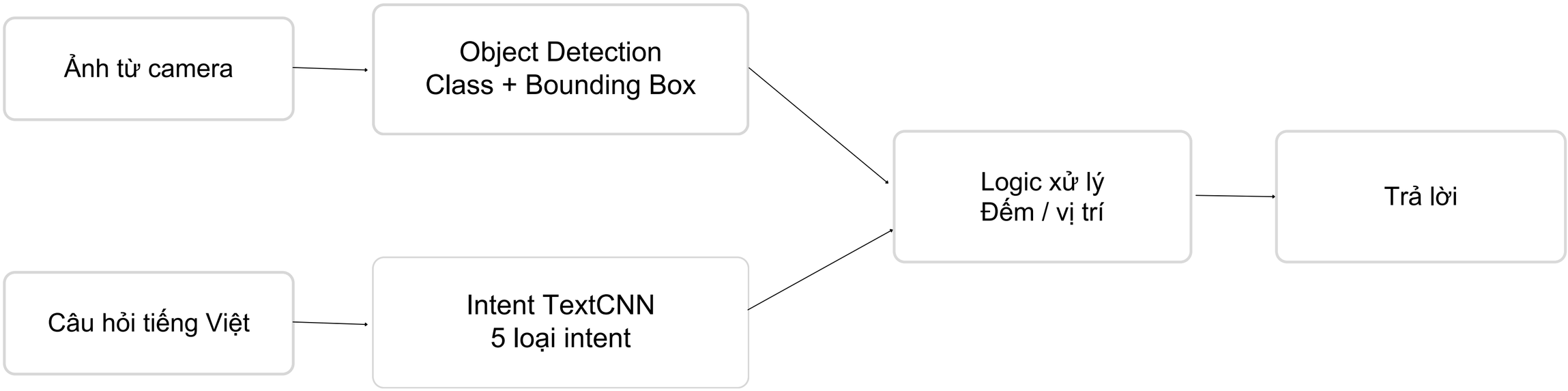
EDGE AI LÀ GÌ?

AI xử lý dữ liệu ngay trên thiết bị — không cần gửi lên cloud.



TỐC ĐỘ PHẢN HỒI NHANH / ÍT PHỤ THUỘC CLOUD / TỐI ƯU CHO THIẾT BỊ NHÚNG

Bài toán của cuộc thi



Vòng loại: chấm hai mô hình trên Kaggle. Chung kết: chạy cả hệ thống trên ESP32-S3.

RÀNG BUỘC KỸ THUẬT

- **Parameters detection:** Tối đa 200.000 tham số
- **Quantization (Model INT8):** Tối đa 1,5 MB
- **Input image:** Tối đa 320 × 240
- **Inference detection:** Không ensemble / TTA
- **Latency:** Chung kết ≤ 10 giây
- **Pretrained:** Được dùng, phải khai báo

Dataset ảnh – 7 lớp vật thể

Bộ nhãn phát hiện vật thể (7 lớp)

bottle	cup	phone	book	mouse	box	toy_car
						
Chai	Cốc	Điện thoại	Sách	Chuột máy tính	Hộp	Ô tô đồ chơi
ID: 0	ID: 1	ID: 2	ID: 3	ID: 4	ID: 5	ID: 6

Dataset câu hỏi – 5 intent

“Có bao nhiêu chai?”	COUNT
“Có điện thoại không?”	EXIST
“Trong ảnh có gì?”	LIST
“Cuốn sách ở đâu?”	LOCATION
“Hôm nay trời có mưa?”	OUT_OF_SCOPE

Dataset & Preprocessing

Ảnh (Detection)

Ảnh 112 × 112

↓

Resize + Normalize

↓

Augmentation (lật, sáng, xoay)

↓

Ảnh + bounding box sau xử lý

Văn bản (TextCNN)

Câu hỏi tiếng Việt

↓

Chuẩn hóa văn bản

↓

Tokenization + Vocabulary

↓

Padding + dãy token ID

CNN BACKBONE | TRÍCH XUẤT ĐẶC TRƯNG



Ảnh đầu vào

112 × 112

TẦNG NÔNG

Cạnh · Góc

Conv2D + ReLU

TẦNG GIỮA

Hình dạng

Stride / Pool

TẦNG SÂU

Ngữ nghĩa

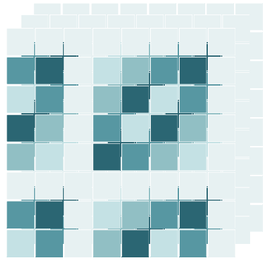
Conv Blocks

FEATURE MAPS

Đa mức, đa tỷ lệ

Các tầng sâu có ngữ nghĩa mạnh hơn; tầng nông giữ chi tiết không gian.

DETECTION HEAD | CLASS + BOUNDING BOX



Feature Maps

Từ Backbone / Neck

DETECTION HEAD

Class

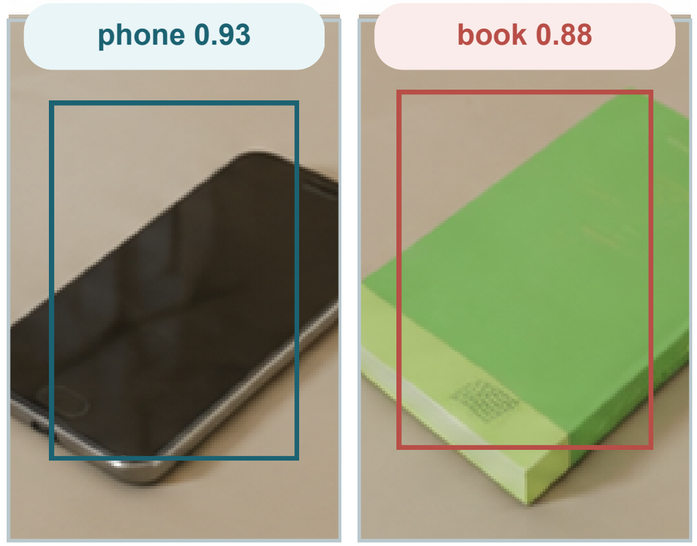
phone / book / bottle

Confidence

0.93 / 0.88

BBox

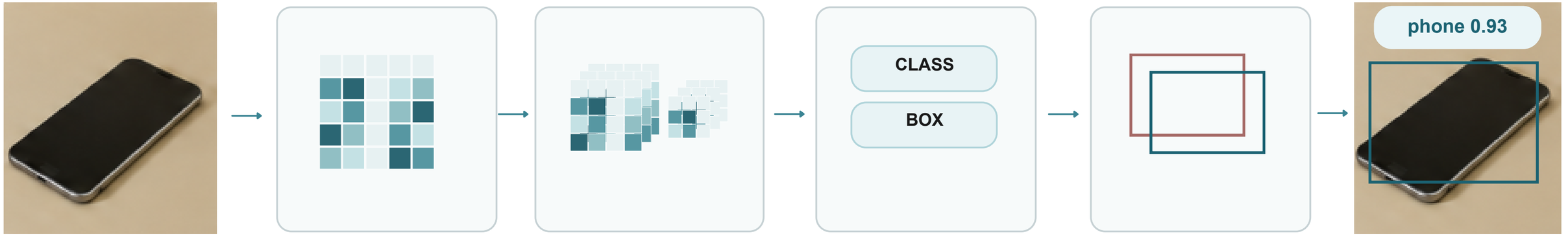
(x, y, w, h)



Ảnh nguồn + bbox minh họa

Head dự đoán lớp và tọa độ cho nhiều vị trí trên feature map.

OBJECT DETECTION | PIPELINE HOÀN CHỈNH



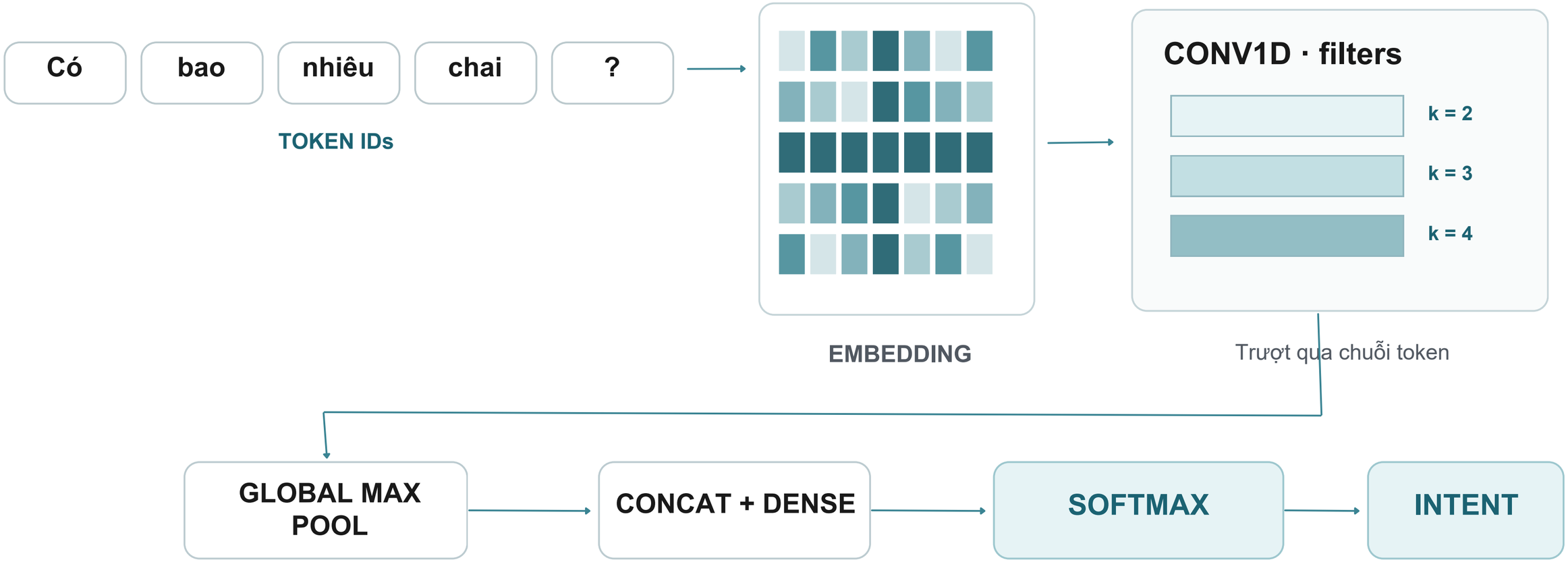
INPUT	BACKBONE	NECK / FPN	HEAD	SCORE + NMS	OUTPUT
Ảnh	Đặc trưng	Đa tỷ lệ	Class + Box	Lọc box	Kết quả

FPN là tùy chọn: mô hình nhỏ có thể bỏ để tiết kiệm RAM và độ trễ.

TEXTCNN | CNN CHO DỮ LIỆU VĂN BẢN

INPUT TEXT

“Có bao nhiêu chai?”



Conv1D trích xuất mẫu cụm từ; không dự đoán bounding box.

INTENT DETECTION | 5 NHÃN CỦA CUỘC THI

CÂU HỎI

“Có bao nhiêu chai?”

TEXTCNN

Xác suất 5 nhãn

COUNT	94%
EXIST	3%
LIST	2%
LOCATION	1%
OUT_OF_SCOPE	0%

COUNT

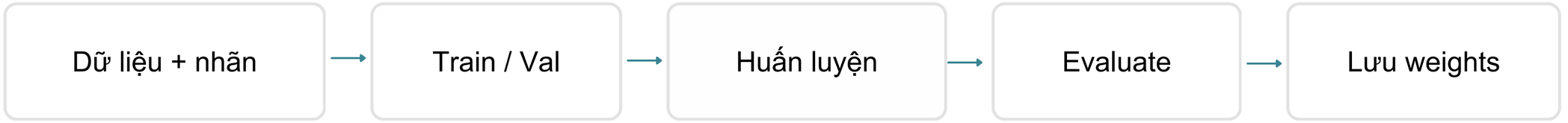
Object Detection

Các bbox có nhãn “chai”



3 chai

Pipeline Training hai mô hình



TextCNN

- Input: token IDs
- Loss: Cross Entropy
- Metric: Accuracy

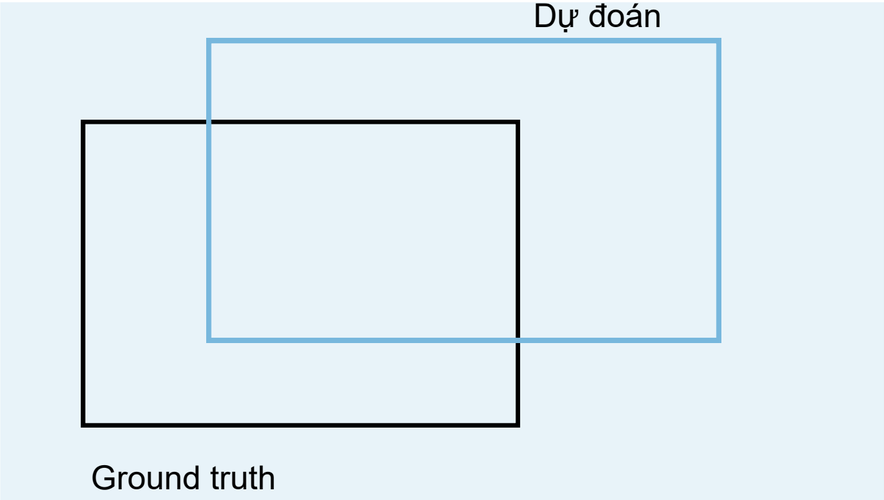
Object Detection

- Input: ảnh + bounding boxes
- Loss: lớp + tọa độ box
- Metric: mAP50

Phân tích lỗi trên validation; không dùng dữ liệu test ẩn để train hoặc pseudo-label.

Evaluate – đo chất lượng mô hình

Detection: IoU và mAP50



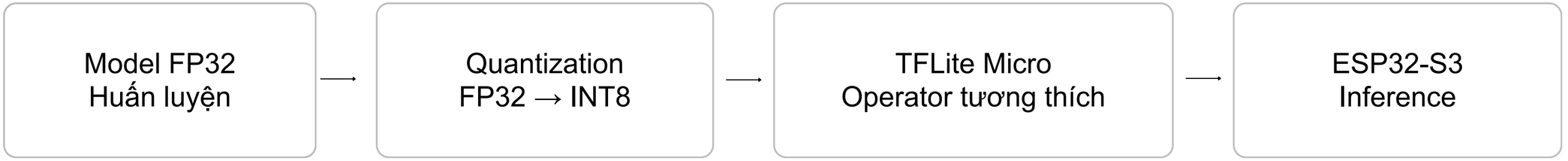
Intent: Accuracy

$$\text{Accuracy} = \frac{\text{Số câu đúng}}{\text{Tổng số câu}}$$

Đúng nhãn trong 5 intent.

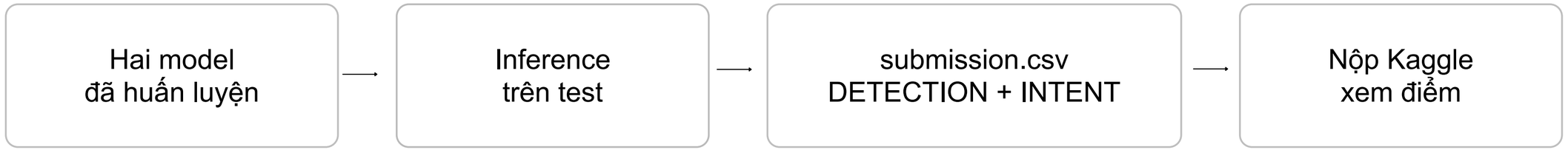
Điểm vòng bảng = 70 × mAP50 + 30 × Accuracy

Quantization INT8 & triển khai Edge



- Giảm kích thước lưu trữ weights.
- Giảm bộ nhớ và chi phí tính toán.
- Kiểm tra sai số sau Quantization.
- Đo latency trên thiết bị thực.

Kaggle Submission & Hands-on



Cột CSV: task_type, sample_id, prediction

- DETECTION: JSON các bbox.
- INTENT: một nhãn trong 5 loại.
- Mỗi sample ID xuất hiện một lần.
- Tối đa 4 submission / 24 giờ.

Demo: load weights → predict → kiểm tra CSV → submit.

TỪ KAGGLE ĐẾN ESP32-S3

Huấn luyện trên PC, suy luận và trả lời trực tiếp trên thiết bị.

01

TRAIN

CNN Detection
Intent TextCNN

→

02

EVALUATE

mAP50, Accuracy
Kiểm tra lỗi

→

03

CONVERT

Quantization INT8
TFLite model

→

04

DEPLOY

ESP-IDF
TFLite Micro

→

05

INFERENCE

Camera + HTTP
JSON trả lời

VÒNG BẢNG

Train + submit trên Kaggle

CHUNG KẾT

Không offload ảnh/câu hỏi để suy luận

PHẦN CỨNG ESP32-S3 N16R8



CPU

2 nhân Xtensa LX7
Tối đa 240 MHz

SRAM

512 KB
Bộ nhớ trong chip

PSRAM

8 MB
RAM ngoài

FLASH

16 MB
Lưu firmware + model

N16R8 = 16 MB Flash + 8 MB PSRAM

SRAM, DRAM, PSRAM VÀ FLASH

SRAM NỘI	**512 KB**	RAM vật lý trong chip Chứa heap nội, stack, buffer nhanh
DRAM	**Một vùng của SRAM**	Vùng địa chỉ dùng cho dữ liệu Không phải chip RAM riêng
PSRAM NGOÀI	**8 MB**	Framebuffer lớn, tensor arena phù hợp Truy cập chậm hơn SRAM nội
FLASH	**16 MB**	Firmware, hằng số và model INT8 Không dùng như RAM cấp phát động

AI TIÊU THỤ RAM NHƯ THẾ NÀO?

Ví dụ bộ đệm ảnh 320 × 240 pixel (chưa gồm overhead):



CÁC VÙNG NHỚ CẦN TÍNH CÙNG LÚC

- Camera framebuffer
- Input tensor
- Tensor Arena
- Stack / Wi-Fi / HTTP

CAMERA OV7670 → INPUT TENSOR



01 **CAPTURE**
Khung hình
320 × 240

02 **RESIZE**
VD: 128 × 128
Đúng model input

03 **QUANTIZE**
Chuẩn hóa + INT8
Đúng scale/zero-point

04 **INVOKE**
Detection
Class + bbox

Preprocessing ở thiết bị phải khớp chính xác với lúc training.

Đấu nối OV7670 theo repo mẫu

DATA BUS	D0–D7	→	GPIO4–GPIO11
ĐỒNG BỘ	XCLK / PCLK	→	GPIO12 / GPIO13
ĐỒNG BỘ	VSYNC / HREF	→	GPIO14 / GPIO15
SCCB	SDA / SCL	→	GPIO16 / GPIO17
NGUỒN	3.3V / GND	→	3V3 / GND
ĐIỀU KHIỂN	RST / PWDN	→	3.3V / GND

Dây ngắn, chung GND, kiểm tra pull-up SDA/SCL. XCLK mặc định 10 MHz; có thể giảm khi nhiễu.

Chạy thử camera với webstream

CỔNG 80

/ → giao diện

/capture → ảnh JPEG

CỔNG 81

:81/stream

MJPEG liên tục

WI-FI

STA theo cấu hình

Thất bại → SoftAP

LUỒNG STREAM

OV7670 → **frame RGB565** → **frame2jpg** → **HTTP** → **trình duyệt**

JPEG được nén bằng phần mềm, tốn CPU và ảnh hưởng tốc độ stream. Dùng một luồng stream khi kiểm thử; xem FPS trong log. Repo thử với ESP-IDF 5.2.6.

Chuyển model: FP32 → INT8

TRƯỚC LƯỢNG TỬ

MODEL FP32

32 bit / trọng số

→

Converter

+ calibration

→

SAU LƯỢNG TỬ

MODEL INT8

8 bit / trọng số

01

Representative dataset

02

Đọc dtype, scale, zero-point

03

Operator tương thích TFLM

04

So sánh Accuracy / mAP50

Không suy ra dtype từ tên file. Với TextCNN, kiểm tra cả kiểu dữ liệu của token IDs.

TFLite Micro: khởi tạo trước khi Invoke

model.cc + model.h
Model

→

GetModel
Kiểm tra schema

→

REGISTER_OPS
Op resolver

→

MicroInterpreter
+ arena

→

AllocateTensors
Sẵn sàng Invoke

MODEL_CONFIG.H

32 × 32 × 3 · UINT8 · 10 lớp

Đây là cấu hình mẫu; cần thay theo model thực tế.

TENSOR ARENA

Mẫu: 200 KiB trong PSRAM

Theo dõi AllocateTensors và mức sử dụng bộ nhớ.

Repo hiện là ví dụ phân loại ảnh qua HTTP; detection bbox và TextCNN cần bổ sung.

Kiểm thử inference qua HTTP

GET /ping

Kiểm tra trạng thái

Trả kích thước input và số lớp của model.

POST /infer

Gửi ảnh raw

Body đúng W × H × C byte; không gửi file JPEG vào endpoint mẫu.

RESPONSE / JSON

```
{
 "class": ...,
 "score": ...,
 "scores": [...],
 "recv_ms": ...,
 "infer_ms": ...,
 "total_ms": ...
}
```

Nếu đổi UINT8 → INT8: sửa đường đọc/ghi tensor và cách xử lý output cho đúng dtype.

Cấu trúc hai project mẫu

esp32_c_infer_script

main/

main.cpp

model.cc / model.h

model_config.h

wifi_config.h

CMakeLists.txt

idf_component.yml

ov7670_esp32s3_webstream

main/

main.c

Kconfig.projbuild

CMakeLists.txt

idf_component.yml

sdkconfig.defaults

Hai project chạy riêng. Khi tích hợp, thống nhất app_main, component, Wi-Fi và HTTP server.

Cấu hình → build → flash → kiểm thử

LỆNH CHẠY TRONG MỖI PROJECT

```
idf.py set-target esp32s3

idf.py menuconfig

idf.py build

idf.py -p <PORT> flash

idf.py -p <PORT> monitor
```

- **Camera: Wi-Fi qua menuconfig**

  Menu “OV7670 Webcam”

- **Inference: wifi_config.h**

  Thay model.cc / model.h và model_config.h

- **Kiểm tra PSRAM và Flash**

  Camera repo mặc định Flash 4 MB; N16R8 có 16 MB.

- **Đọc IP và kiểm thử endpoint**

  /ping, /infer; hoặc /capture, :81/stream

> Giữ dependency và cấu hình build có thể tái tạo; lần build đầu cần tải component.

Ghép detection + TextCNN: phần cần bổ sung

/ask, tokenizer, hai model và logic hỏi đáp chưa có sẵn trong hai repo mẫu.

CÂU HỎI

POST /ask
Endpoint đề xuất

→

TEXTCNN + TỪ ĐIỂN

Intent + lớp vật thể

CAMERA

OV7670 → frame thô

→

DETECTION

Preprocess → Invoke
Decode / NMS

LOGIC

Đếm / tồn tại
Danh sách / vị trí

→

OUTPUT

JSON trả lời

Mục tiêu chung kết: toàn bộ suy luận ở ESP32-S3; không offload ảnh/câu hỏi sang server.

Checklist trước khi chạy toàn hệ thống

01 / CHẤT LƯỢNG

PC ↔ thiết bị

Đối chiếu output; đo mAP50 và Accuracy sau lượng tử.

02 / BỘ NHỚ

Heap + arena + buffer

Kiểm tra heap trống, khối trống lớn nhất và bộ nhớ đỉnh.

03 / ĐỘ TRỄ

Tổng phản hồi ≤ 10 giây

Đo capture, tiền xử lý, inference và logic riêng.

04 / ỔN ĐỊNH

Nhiều lượt liên tiếp

Test OOS, mất Wi-Fi; trả framebuffer và free JPEG.

Test từng nhánh trước → ghép hệ thống → ghi log → chốt cấu hình và bản build.